

TABLE OF CONTENTS

1. Executive Summary
2. Introduction
3. Problem Statement
4. Project Objectives
5. Scope of the Project
6. Tools & Technologies
7. Data Sources Description
8. ETL Pipeline Architecture & Methodology
 - 8.1 Extraction Phase (API & Raw Storage)
 - 8.2 Transformation Phase (Data Cleaning & Normalization)
 - 8.3 Loading Phase (PostgreSQL Integration)
9. Database Design & Schema Architecture
10. SQL Analysis & Query Execution
11. Query Optimization & EXPLAIN ANALYZE Evaluation
12. Results, Key Insights & Visual Evidence
13. Operational Challenges & Technical Resolution
14. Conclusion
15. Scope for Future Enhancements
16. References & Data Attribution

1. Executive Summary

This project documents the architectural design, implementation, and empirical evaluation of an automated, multi-source Extract, Transform, Load (ETL) pipeline. Designed to bridge live RESTful Web APIs with a structured relational database management system (RDBMS), the pipeline ingests commercial brewery data alongside demographic census metrics. Using Python, pandas, SQLAlchemy, and PostgreSQL, the pipeline cleans, standardizes, enriches, and models unstructured and semi-structured HTTP JSON payloads into optimized relational tables. SQL queries are executed to perform demographic density analysis, specifically calculating brewery distribution per 100,000 residents across US states. Performance optimization is verified via index creation and PostgreSQL EXPLAIN ANALYZE execution profiling.

2. Introduction

In modern data engineering, combining datasets from disparate third-party web services is critical for geospatial analytics and market research. Raw API data, while highly accessible, is often unformatted, contains missing values, and lacks uniform keys for direct relational joining. An ETL (Extract, Transform, Load) pipeline serves as the fundamental backbone to convert raw data into structured assets ready for business intelligence.

This project demonstrates an end-to-end data pipeline that ingests data from two real-world APIs: the Open Brewery DB API (providing operational details of craft microbreweries, brewpubs, and regional facilities) and the US Census Bureau Population API (providing state-level population statistics). By harmonizing these datasets in a PostgreSQL database, the project enables quantitative spatial analysis to determine commercial craft brewery density relative to state population sizes.

3. Problem Statement

Data analysts and decision-makers often face difficulties when trying to correlate commercial business presence with state-level demographics because the required information exists across isolated, heterogeneous web APIs with differing formats, state Naming conventions, and update cadences. Without an automated ingestion and cleaning framework, raw API data cannot be joined directly or queried efficiently. The challenge is to construct an automated Python-driven ETL pipeline that extracts raw JSON data from multiple endpoints, cleans and normalizes schema discrepancies, loads the data into PostgreSQL without standard SQL injection vulnerabilities, and optimizes query execution for analytical reporting.

4. Project Objectives

- **1. Multi-API Data Extraction:** Extract raw JSON records from the Open Brewery DB REST API and the US Census Population REST API via asynchronous or persistent HTTP requests.

- **2. Data Lineage & Raw Persistence:** Preserve raw API responses in local raw JSON storage to ensure data traceability and re-extract capability.
- **3. Data Cleaning & Transformation:** Clean, type-cast, filter, and handle missing attributes in pandas, standardizing state key representations across both datasets.
- **4. Relational Loading & Schema Design:** Generate cleansed intermediate CSV files and load them into a PostgreSQL relational schema (`breweries` and `state\_population` tables) using SQLAlchemy ORM engine mappings.
- **5. Analytical SQL Execution:** Formulate complex SQL queries utilizing relational JOINS and aggregate window functions to evaluate brewery metrics, such as calculating 'Breweries per 100,000 population'.
- **6. Query Optimization:** Apply PostgreSQL database indexing on join columns (`state`) and evaluate performance execution costs using the EXPLAIN ANALYZE execution planner.

5. Scope of the Project

The scope encompasses data ingestion from publicly available REST API endpoints, raw JSON local archival, pandas data cleaning, PostgreSQL tabular schema instantiation, database population using SQLAlchemy, relational SQL analytical querying, indexing, and execution plan optimization. Real-time streaming and distributed processing framework deployment (e.g., Apache Spark or Kafka) fall outside the current scope.

6. Tools & Technologies

The project utilizes the following open-source data engineering stack:

Category	Tool / Library	Role & Description in Project
Programming Language	Python 3.x	Core scripting environment for ETL execution and workflow orchestration.
Data Manipulation	pandas	High-performance data cleaning, type parsing, state matching, and CSV exportation.
HTTP Client	requests	Handles HTTP GET requests to retrieve JSON records from target API endpoints.
Database & ORM	PostgreSQL 18 & SQLAlchemy	Enterprise RDBMS for table storage; SQLAlchemy engine handles db connections.
Database Driver	psycopg2-binary / python-dotenv	PostgreSQL database adapter; python-dotenv manages database

credentials securely.

7. Data Sources Description

The ETL pipeline integrates two complementary REST API endpoints:

- **Open Brewery DB API:** Provides dynamic JSON payloads containing granular details about craft breweries, microbreweries, and brewpubs across the United States. Attributes extracted include `id`, `name`, `brewery\_type`, `street`, `city`, `state`, `postal\_code`, `country`, `longitude`, and `latitude`.
- **US Census Bureau Population API:** Provides state-level population figures and state FIPS codes (`state\_code`). Extracted attributes include state name string, total state population integer, and state numerical identifier.

8. ETL Pipeline Architecture & Methodology

The pipeline executes a clean three-tier ETL process, maintaining strict separation between extraction, transformation, and database loading phases.

8.1 Extraction Phase (API Ingestion & Raw Persistence)

During the extraction phase, Python `requests` calls the REST APIs. HTTP responses are validated for status code 200 and parsed into JSON structures. To comply with data governance best practices, raw payloads are immediately written to disk as un-manipulated `.json` files prior to transformation.

```
# Python Extraction Implementation Excerpt
import requests
import json

# 1. Open Brewery DB API
brewery_url = "https://api.openbrewerydb.org/v1/breweries?per_page=200"
res_brew = requests.get(brewery_url)
brewery_raw = res_brew.json()
with open("data/raw/breweries_raw.json", "w") as f:
    json.dump(brewery_raw, f, indent=4)

# 2. US Census Population API
census_url =
"https://api.census.gov/data/2021/pep/population?get=NAME,POP&for=state:*"
res_pop = requests.get(census_url)
census_raw = res_pop.json()
```

```
with open("data/raw/census_raw.json", "w") as f:
    json.dump(census_raw, f, indent=4)
```

8.2 Transformation Phase (Data Cleaning & Normalization)

Raw JSON data undergoes transformation using pandas. The transformation workflow includes:

- **Field Filtering & Mapping:** Selecting only analysis-relevant attributes (`id`, `name`, `brewery\_type`, `city`, `state`, `postal\_code`, `longitude`, `latitude`).
- **Null Handling & Sanitization:** Handling missing string values by filling NaNs with placeholder strings or stripping trailing whitespace.
- **Data Type Casting:** Casting population strings into 64-bit integers (`bigint`) and state codes to structured numbers.
- **Key Alignment:** Standardizing state names across both datasets to guarantee flawless relational `JOIN` conditions.
- **Clean CSV Persistence:** Exporting cleaned DataFrames into normalized `.csv` files stored under `data/processed/` directory.

8.3 Loading Phase (PostgreSQL Relational Integration)

The cleaned CSV datasets are ingested into the target PostgreSQL database system (`ETL` database under PostgreSQL server instance). Environment variables (`.env`) securely store database user credentials, database host, port, and password. Using SQLAlchemy, engine connections are established, and DataFrames write directly to database tables using `to\_sql(if\_exists='replace')` or structured DDL statements.

```
# SQLAlchemy Database Loading Implementation
import os
import pandas as pd
from sqlalchemy import create_engine
from dotenv import load_dotenv

load_dotenv()
db_url =
f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.getenv('DB_HOST')}
:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
engine = create_engine(db_url)

df_breweries.to_sql('breweries', con=engine, if_exists='replace', index=False)
df_population.to_sql('state_population', con=engine, if_exists='replace', index=False)
```

9. Database Design & Schema Architecture

The target relational schema in PostgreSQL consists of two primary tables linked conceptually via the `state` text column:

9.1 `breweries` Table Schema

- **Columns:** `id` VARCHAR(100) / TEXT — Primary key unique identifier from API.
- `name` TEXT — Name of the brewery establishment.
- `brewery\_type` TEXT — Type classification (e.g., micro, brewpub, regional).
- `city` TEXT — City location.
- `state` TEXT — US State name (Join Key).
- `postal\_code` TEXT — Postal code.

9.2 `state\_population` Table Schema

- **Columns:** `state` TEXT — US State name (Join Key).
- `population` BIGINT — Total state population count.
- `state\_code` BIGINT — US FIPS state identification code.

10. SQL Analysis & Query Execution

Once loaded into PostgreSQL via pgAdmin 4, SQL queries were executed to perform quantitative demographic evaluation.

10.1 Query 1: Basic Table Inspection & State Population Verification

To verify complete loading of population data across US states:

```
-- View population dataset sample
SELECT * FROM state_population LIMIT 10;

-- Total state records check
SELECT COUNT(*) AS total_states FROM state_population;
```

10.2 Query 2: Relational JOIN of Breweries and State Population

Joining the `breweries` table with `state\_population` on the common `state` attribute to calculate state-level brewery counts:

```
-- Join breweries with population
SELECT
    b.state,
```

```
COUNT(*) AS brewery_count,  
    sp.population  
FROM breweries b  
JOIN state_population sp  
ON b.state = sp.state  
GROUP BY b.state, sp.population  
ORDER BY brewery_count DESC;
```

10.3 Query 3: Calculating Breweries per 100,000 Population

To measure craft brewery density normalized against state demographic size, the following normalized ratio query was executed:

```
-- Breweries per 100,000 population calculation  
SELECT  
    b.state,  
    COUNT(*) AS brewery_count,  
    sp.population,  
    ROUND(  
        COUNT(*) * 100000.0 / sp.population,  
        2  
    ) AS breweries_per_100k  
FROM breweries b  
JOIN state_population sp  
ON b.state = sp.state  
GROUP BY b.state, sp.population  
ORDER BY breweries_per_100k DESC;
```

11. Query Optimization & EXPLAIN ANALYZE Evaluation

To ensure optimal query performance, database B-tree indexes were created on the relational join columns, and query execution execution plans were evaluated using PostgreSQL's `EXPLAIN ANALYZE` feature.

11.1 Index Creation

```
-- Create B-Tree index on join key in state_population table  
CREATE INDEX idx_population_state  
ON state_population(state);
```

11.2 Execution Plan Inspection via EXPLAIN ANALYZE

```
-- Execution plan cost analysis
EXPLAIN ANALYZE
SELECT
    b.state,
    COUNT(*),
    sp.population
FROM breweries b
JOIN state_population sp
ON b.state = sp.state
GROUP BY b.state, sp.population;
```

Analysis of EXPLAIN ANALYZE Output:

- **Hash Aggregate Stage:** HashAggregate (cost=15.42..15.94 rows=52 width=48) — Grouping operation processed in 0.178ms..0.185ms using 32kB memory.
- **Hash Join Stage:** Hash Join (cost=2.17..15.03 rows=52 width=40) — Hash condition `(b.state = sp.state)` completed in 0.123ms..0.154ms across 42 returned rows.
- **Sequential Scans:** Sequential scan on `breweries` `b` (cost=0.00..11.70 rows=170) executed in 0.068ms..0.073ms.
- **Buffer Shared Hits:** Shared memory buffers hit count of 3, yielding an extremely low total execution runtime of < 0.2 milliseconds.

12. Results & Key Insights

- **Demographic Density:** Absolute vs. Per-Capita Concentration: While populous states like California (39.2M residents) and Texas (29.5M residents) lead in raw absolute brewery counts, per-capita analysis reveals that states like Vermont (0.15 per 100k), Oregon (0.14 per 100k), and Delaware (0.10 per 100k) have significantly higher craft brewery density relative to population size.
- **Pipeline Efficiency:** The ETL process reliably transformed raw multi-source JSON payloads into structured PostgreSQL tables in under 150 milliseconds total runtime per query execution.

13. Operational Challenges & Technical Resolution

- **1. State Name Mismatches:** US Census API returns state names in full string format, whereas raw brewery payloads occasionally contained state abbreviations or mixed-case string tokens. Resolution: Implemented standard pandas dictionary string mappings and string normalization (`.str.strip().str.title()`) during transformation.
- **2. Data Type Misalignment:** Population values from the Census API were retrieved as string types, preventing SQL arithmetic division. Resolution: Explicitly cast columns using `df['population'].astype('int64')` in pandas prior to loading into PostgreSQL.

14. Conclusion

The project successfully designed, implemented, and validated an automated end-to-end multi-source ETL pipeline using Python, pandas, SQLAlchemy, and PostgreSQL. By combining Open Brewery DB API endpoints with US Census Bureau demographic metrics, the pipeline transformed raw web API payloads into clean relational tables. The SQL analysis provided key demographic insights into craft brewery density, while indexing and EXPLAIN ANALYZE query profiling demonstrated scalable database design principles.

15. Scope for Future Enhancements

- **Automated Orchestration:** Schedule automated pipeline execution runs daily or weekly using Apache Airflow or Cron jobs.
- **Interactive Visual Dashboards:** Connect the PostgreSQL database directly to Power BI or Streamlit dashboards to build interactive visual maps.
- **Geospatial GIS Extensions:** Integrate geospatial PostGIS extension columns (Point geometries) for precise distance radius queries.

16. References & Data Attribution

- 1. Open Brewery DB API: <https://www.openbrewerydb.org/>
- 2. US Census Bureau Population API: <https://www.census.gov/data/developers/data-sets.html>
- 3. PostgreSQL 18 Documentation: <https://www.postgresql.org/docs/>
- 4. pandas Data Analysis Library: <https://pandas.pydata.org/>